

Лекция 2. Системы управления базами данных (СУБД)

Дисциплина: Методы и средства проектирования баз данных

Составитель: Монастырев Егорий Егорьевич, М-ВТ-25

Якутск, 2026

На Лекции 1 мы установили:

- ▶ данные существуют в разных форматах и организуются по разным моделям представления (табличной, документной, файловой, векторной);
- ▶ предметная область описывается с помощью ER-модели: сущности, атрибуты, связи;
- ▶ выбор способа хранения — результат анализа данных, а не исходная точка проектирования.

Открытый вопрос

Мы говорили, что данные находятся **под управлением специализированного программного обеспечения**. Что это за программное обеспечение, зачем оно нужно и как оно устроено?

1. Чем база данных отличается от СУБД, а СУБД — от системы базы данных?
2. Какие функции обязана выполнять СУБД и зачем каждая из них нужна?
3. Что такое трёхуровневая архитектура ANSI/SPARC и что такое независимость данных?
4. Из каких программных компонентов состоит СУБД изнутри?
5. Как устроена клиент-серверная модель работы с СУБД?
6. Какие роли участвуют в работе с СУБД?
7. Чем настольная СУБД отличается от серверной?
8. Из каких подязыков состоит SQL и с какими функциями СУБД они связаны?
9. Что такое индекс и зачем он нужен, на уровне идеи?

База данных $\rightarrow$ СУБД $\rightarrow$ Система базы данных

Почему одной базы данных недостаточно

Представим, что данные просто лежат в файлах на диске, без управляющей программы.

- ▶ Кто гарантирует, что два сотрудника не испортят одну и ту же запись, редактируя её одновременно?
- ▶ Кто восстановит данные после сбоя электропитания посреди записи?
- ▶ Кто проверит, что оценка не выставлена несуществующему студенту?
- ▶ Кто ограничит доступ так, чтобы студент не мог править чужие оценки?

Вывод

Сами данные не решают эти задачи. Ими управляет специализированное программное обеспечение — **СУБД**.

Неполное, бытовое определение

«База данных — структурированный набор данных.»

Проблема такого определения

Под него подходит и обычная таблица Excel или CSV-файл — но это не база данных в строгом смысле.

Уточнённое определение

База данных — организованная в соответствии с определённой **схемой** совокупность взаимосвязанных данных, отражающая состояние некоторой предметной области, предназначенная для совместного использования многими приложениями и пользователями и находящаяся под управлением специализированного программного обеспечения.

Неполное, бытовое определение

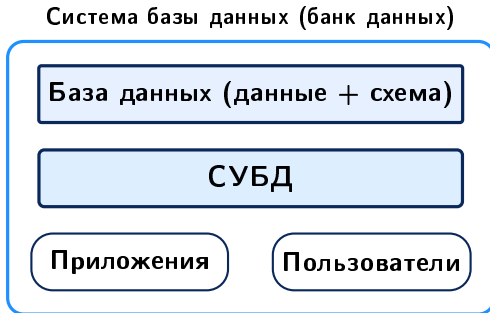
«СУБД — программа для создания, хранения и обработки БД.»

Проблема такого определения

Такая формулировка не объясняет, **зачем** именно нужна СУБД, и создаёт впечатление, что это просто «редактор таблиц».

Уточнённое определение

СУБД (DBMS) — комплекс программных средств, обеспечивающий определение схемы, создание, заполнение, модификацию и контролируемый доступ к базам данных, а также реализующий обработку транзакций, поддержание целостности, разграничение доступа, журналирование и восстановление данных после сбоев.



Определение

Система базы данных — совокупность базы данных, СУБД, прикладных программ и пользователей, работающих с этой базой данных как единым целым.

Понятие	Что это
База данных	данные + схема
СУБД	программа управления данными
Система базы данных	данные + схема + СУБД + приложения + люди
Информационная система	система БД + бизнес-логика + интерфейсы + интеграции

На примере «Университета»

Данные о студентах и группах — **база данных**; PostgreSQL, под управлением которого они хранятся, — **СУБД**; портал деканата, объединяющий БД, СУБД, приложение и сотрудников, — **система базы данных**.

Зачем нужна СУБД и что именно она делает?

Общее назначение СУБД

СУБД решает задачи, которые невозможно надёжно решить средствами отдельных файлов и прикладных программ:

- ▶ предоставляет единый, контролируемый способ доступа к данным;
- ▶ берёт на себя ответственность за корректность данных при сбоях и параллельной работе;
- ▶ скрывает от приложений детали физического хранения данных.

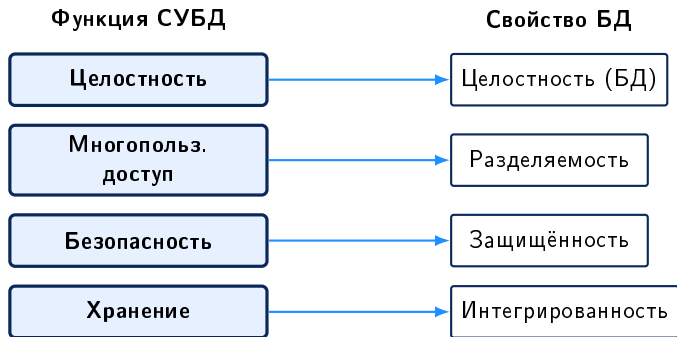
Идея

Приложения работают с данными **через** СУБД, а не напрямую с файлами на диске.

Функция	Суть
Хранение данных	организация данных на диске в виде таблиц, файлов, страниц
Обработка и оптимизация запросов	разбор и эффективное выполнение запросов к данным
Управление транзакциями	выполнение операций по принципу «всё или ничего»
Контроль целостности	проверка правил, которым должны удовлетворять данные
Управление многопользовательским доступом	корректная одновременная работа многих пользователей
Журналирование и восстановление	защита от потери данных при сбоях
Безопасность	разграничение прав доступа к данным

- ▶ **Целостность** — нельзя выставить оценку студенту с несуществующим номером;
- ▶ **Многопользовательский доступ** — два сотрудника деканата одновременно редактируют оценки одного студента, и это не приводит к потере данных;
- ▶ **Журналирование и восстановление** — при сбое питания посреди сохранения ведомости данные не теряются;
- ▶ **Безопасность** — студент видит только свои оценки, а не оценки всей группы;
- ▶ **Транзакции** — перевод студента из одной группы в другую либо выполняется полностью, либо не выполняется вовсе.

От функций СУБД — к свойствам базы данных



Главная мысль

Свойства базы данных, названные в Лекции 1 (целостность, разделяемость, защищённость, интегрированность), — не сами по себе, а **результат** работы конкретных функций СУБД.

Как одни и те же данные видны на разных уровнях абстракции?

- ▶ **Схема базы данных** — формальное описание структуры БД: таблицы, атрибуты, типы данных, связи, ограничения. Меняется редко.
- ▶ **Экземпляр базы данных** — конкретное состояние данных в определённый момент времени. Меняется практически при каждой операции записи.

Аналогия для запоминания

Схема — это «тип» (структура), экземпляр — это «значение» (конкретное содержимое) в данный момент.

Пример: схема таблицы `student` (`id`, ФИО, `group_id`) не меняется годами; строки этой таблицы меняются каждый день.

Трёхуровневая архитектура ANSI/SPARC

Это не этапы проектирования, а архитектурная модель самой СУБД: способ, которым данные одновременно представлены на трёх уровнях.



Цель — независимость данных.

- ▶ **Внешний** — как данные видит конкретный пользователь или приложение (студент видит свои оценки, деканат — статистику факультета).
- ▶ **Концептуальный** — единое логическое описание всей БД: сущности, атрибуты, связи, ограничения.
- ▶ **Внутренний** — как данные физически хранятся: файлы, страницы, индексы, буферы.

Уточнение

Соответствие «внешний уровень $\approx$ VIEW в PostgreSQL» — лишь приближённое: полноценный внешний уровень формируется также правами доступа (GRANT/REVOKE) и структурой запросов приложения, а не только объектами VIEW.

Тип	Что позволяет
Физическая	изменять способы хранения и индексы без переписывания приложений
Логическая	изменять концептуальную схему, сохраняя работоспособность существующих представлений

Важная оговорка

Логическая независимость имеет пределы: удаление или переименование атрибута, который уже использовало приложение, всё равно нарушит его работу.

Из чего состоит СУБД изнутри?

Важное разграничение

Архитектура ANSI/SPARC отвечает на вопрос «**как данные видны снаружи**».

Внутренняя архитектура СУБД отвечает на вопрос «**из каких программных модулей СУБД состоит изнутри**».

Это разные по природе понятия, которые не стоит путать: первое — про уровни абстракции представления данных, второе — про устройство программного ядра.

Компонент	Что делает
Обработчик запросов Менеджер транзакций	разбор и оптимизация запросов контроль атомарности и изоляции операций
Менеджер хранения и буфера	управление файлами, страницами, кэшем в ОЗУ
Менеджер журнала и восстановления	фиксация изменений, восстановление после сбоев
Словарь данных / системный каталог	хранение метаданных о структуре БД

Каждый компонент отвечает за одну из функций СУБД, рассмотренных в разделе 2.

Путь одного запроса через компоненты СУБД

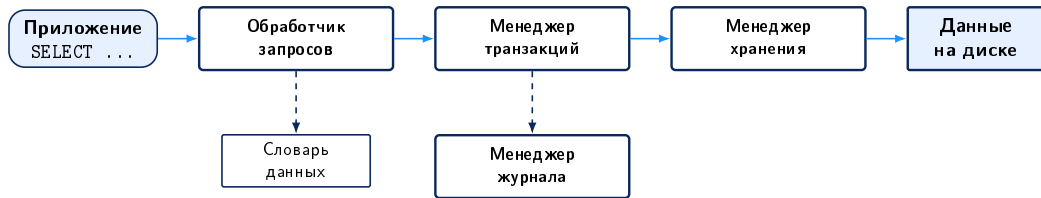
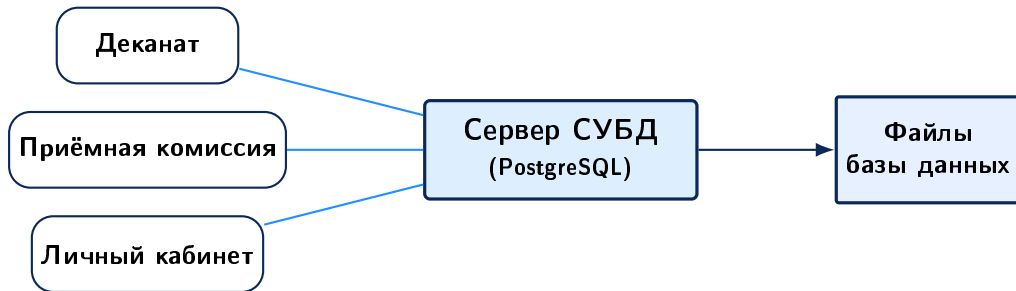


Схема иллюстрирует общий путь запроса без разбора алгоритмов оптимизации: обработчик обращается к словарю данных, чтобы узнать структуру таблиц; менеджер транзакций взаимодействует с менеджером журнала для гарантии устойчивости изменений.

Как СУБД обслуживает многих пользователей одновременно?

Клиент-серверная модель доступа к СУБД



Идея

Множество клиентских приложений одновременно подключаются к **одному** серверу СУБД, а не к файлам напрямую.

Сотрудник А редактирует оценку студента 101

Сотрудник Б **одновременно** редактирует ту же оценку

- ▶ Что произойдёт, если оба сохраняют изменения одновременно?
- ▶ Как СУБД не допускает потери одного из изменений?

На этом этапе — только постановка вопроса

Сама механика решения (блокировки, уровни изоляции) рассматривается в отдельной будущей лекции о транзакциях и параллельном выполнении. Сейчас важно зафиксировать: СУБД обязана это контролировать.

Роль	Что делает
Администратор БД (АБД)	администрирует безопасность, резервное копирование и восстановление, следит за производительностью
Прикладные программисты Конечные пользователи	используют транзакции и целостность через приложение, не работая с ядром СУБД напрямую взаимодействуют с данными через приложение, не видя внутреннего устройства СУБД

Какие бывают СУБД и как к ним подключаются?

Классификация СУБД по модели данных (напоминание)

Из Лекции 1: способ представления данных определяет и тип СУБД, работающей с этими данными.

Реляционные	Документные	Другие
PostgreSQL, MySQL, Oracle	MongoDB	Vector DB и т.д.

Далее в этой лекции

Речь пойдёт о другой оси классификации — не о модели данных, а об архитектуре развёртывания СУБД.

Настольные и серверные СУБД

	Access	PostgreSQL
Тип	настольная СУБД	серверная СУБД
Интерфейс	сильный GUI	SQL + инструменты
Масштаб	небольшие системы	крупные системы
Многопольз. доступ	ограничен	полноценный

Вывод

Access удобен для локальных обучающих задач; PostgreSQL — основная серверная СУБД курса.

Для подключения к серверной СУБД используются отдельные клиентские инструменты:

- ▶ **pgAdmin** — графическое администрирование PostgreSQL: просмотр таблиц, выполнение запросов, настройка прав доступа;
- ▶ **psql** — консольный (CLI) клиент PostgreSQL для прямого взаимодействия с сервером.

Важно

Это средства **работы** с уже существующей серверной СУБД, а не средства проектирования схемы БД (те рассматривались в Лекции 1: draw.io, pgModeler, DBeaver).

Как практически формулируется команда для работы с СУБД?

Определение

SQL (Structured Query Language) — стандартизированный декларативный язык для определения структуры данных, выполнения операций над данными и управления доступом в реляционных СУБД.

- ▶ декларативный — пользователь описывает, **что** нужно получить, а не **как** именно это будет вычислено;
- ▶ поддерживается практически всеми реляционными СУБД (с диалектными отличиями).

На этом этапе курса

Подробный синтаксис запросов не рассматривается — только структура языка и её связь с функциями СУБД.

Подъязык	Назначение	Пример команд
DDL	определение структуры данных	CREATE, ALTER, DROP
DML	работа с данными	SELECT, INSERT, UPDATE, DELETE
DCL	управление доступом	GRANT, REVOKE
TCL	управление транзакциями	COMMIT, ROLLBACK

Главная идея

SQL — не хаотичный набор команд, а язык, структурно повторяющий функции самой СУБД: хранение и структуру (DDL), обработку данных (DML), безопасность (DCL), транзакции (TCL).

Где физически лежат данные, с которыми работает SQL?

От логической модели к физическому хранению



Пользователь и приложение работают только с логической структурой; всё, что ниже, скрыто внутренним уровнем архитектуры ANSI/SPARC (раздел 3).

- ▶ **Файл** — единица хранения таблицы или её части на диске;
- ▶ **Страница** — блок фиксированного размера, которым СУБД читает и записывает данные на диск;
- ▶ **Буфер** — область оперативной памяти, где хранятся недавно использованные страницы, чтобы не обращаться к диску повторно.

Важно

Это обзорное представление. Алгоритмы работы буфера и физической организации файлов — предмет отдельной будущей лекции о физическом проектировании.

Поиск студента по фамилии в таблице из миллиона строк:

- ▶ **без индекса** — СУБД просматривает все строки подряд;
- ▶ **с индексом** — СУБД использует вспомогательную структуру, которая сразу указывает, где искать нужные строки.

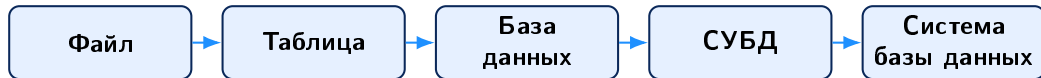
Определение

Индекс — вспомогательная структура данных, ускоряющая поиск записей ценой дополнительного объёма хранения и дополнительных затрат при изменении данных.

Синтаксис создания индекса и стратегии индексирования — предмет отдельной будущей лекции.

От файла — к системе управления базами данных

Итоговая схема: от файла к системе базы данных



Главная идея лекции

СУБД — не просто программа для хранения таблиц, а комплекс функций (транзакции, целостность, многопользовательский доступ, восстановление, безопасность), реализованных набором программных компонентов и доступных через SQL.

Что будет раскрыто подробнее в следующих лекциях

Тема	Где
Реляционная модель, ключи, ограничения целостности (SQL)	Лекция 3
Нормализация и нормальные формы	Лекция 3 / отдельная лекция
Полный синтаксис SQL, JOIN, агрегатные функции	SQL-практикум
Механика транзакций: изоляция, блокировки	Отдельная лекция о транзакциях
Детали журналирования и восстановления	Та же лекция о надёжности
Синтаксис и стратегии индексирования	Лекция о физическом проектировании

1. В чём разница между базой данных, СУБД и системой базы данных?
2. Почему определение «СУБД — программа для работы с БД» неполно?
3. Что различают схема и экземпляр базы данных?
4. Из каких трёх уровней состоит архитектура ANSI/SPARC?
5. Чем архитектура ANSI/SPARC отличается от внутренней архитектуры СУБД?
6. Назовите пять программных компонентов ядра СУБД.
7. Что такое клиент-серверная модель работы с СУБД?
8. Чем отличаются DDL, DML, DCL и TCL?
9. Зачем нужен индекс и какова его цена?

Задание к следующей лекции

Возьмите предметную область и ER-диаграмму, разработанную к Лекции 1.

Определите:

1. какие функции СУБД будут задействованы при работе с вашими данными (целостность, многопользовательский доступ и т. д.) — приведите по одному примеру на функцию;
2. какие сущности вашей предметной области логично превратить в таблицы реляционной СУБД.

Результат: короткий список функций СУБД с примерами + перечень будущих таблиц.

Спасибо за внимание!